

DEIN PYTHON VORKURS

Probleme verstehen · Ideen planen · Lösungen bauen

10 Kurstage · 60 Aufgaben · dein eigenes Mini-Projekt

Für wen?

Für Teilnehmende ohne oder mit wenig Programmiererfahrung. Pro Kurstag sind etwa 90–120 Minuten vorgesehen.

Arbeitsweise

Erst vorhersagen und planen, dann selbst schreiben, testen und verbessern. Musterlösungen sind bewusst nicht enthalten: Der Lösungsweg zählt.

Name: _____

Kurs / Gruppe: _____

So löst du Probleme mit diesem Heft

Python ist dein Werkzeug - aber der wichtigste Teil passiert vor dem Tippen. Du lernst, ein Problem zu verstehen, es in kleine Schritte zu zerlegen und deine Lösung mit guten Beispielen zu prüfen. Eine Aufgabe gilt nicht als fertig, nur weil der Code einmal läuft.

Dein Problemlöse-Kompass

1. Verstehen: Was ist gegeben und was wird gesucht? 2. Planen: Zerlege den Weg in kleine Schritte. 3. Bauen: Übersetze immer nur den nächsten Schritt in Python. 4. Prüfen: Teste Normalfälle, Grenzen und Fehler. 5. Verbessern: Mache Namen, Meldungen und Aufbau klarer.

Die Tutorperson hilft dir beim Denken - nicht nur beim Tippen. Zeige daher auch deine Beispiele, Skizzen, Tabellen und Testideen. Mehrere Lösungswege können richtig sein.

Wenn etwas nicht funktioniert

Vergleiche zuerst Erwartung und tatsächliches Ergebnis. Finde die kleinste Stelle, an der beides auseinandergeht. Lies dann die letzte Zeile der Fehlermeldung und ändere nur eine Sache auf einmal.

Ein Beispiel - noch ohne Code

Problem: Ein Kino verkauft Tickets für 8 Euro. Kinder unter 12 zahlen 5 Euro. Ab drei Tickets gibt es insgesamt 2 Euro Rabatt. Berechne den Gesamtpreis.

1 · Verstehen	Eingabe: Alter und Anzahl der Tickets. Gesucht: der Gesamtpreis. Wichtige Grenzen: Alter 11/12 und Anzahl 2/3.
2 · Mit Beispielen denken	10 Jahre, 2 Tickets → 10 Euro. 20 Jahre, 2 Tickets → 16 Euro. 10 Jahre, 3 Tickets → 13 Euro. 20 Jahre, 3 Tickets → 22 Euro.
3 · In Schritte zerlegen	a) Ticketpreis anhand des Alters bestimmen. b) Preis mit Anzahl multiplizieren. c) Ab drei Tickets 2 Euro abziehen. d) Ergebnis ausgeben.
4 · Prüfen	Was passiert bei genau 12 Jahren? Was passiert bei 0 Tickets? Sind negative Werte erlaubt? Die Beispiele werden zu Testfällen.

Erst jetzt würdest du die einzelnen Schritte in Python übersetzen. Wenn der Code später falsch ist, kannst du jeden Schritt mit deinen Beispielen vergleichen.

Kursübersicht

Tag	Thema	Kernidee
1	Erste Programme: Ausgabe und Syntax	Anweisungen von oben nach unten lesen, Text ausgeben, Kommentare verwenden.
2	Variablen und Datentypen	Werte speichern, verändern und als int, float, str oder bool einordnen.
3	Eingabe und Operatoren	input() verwenden, Texte in Zahlen umwandeln und Rechenoperatoren sicher einsetzen.
4	Bedingungen: if, elif, else	Vergleiche formulieren und genau einen passenden Programmzweig ausführen.
5	Schleifen: for und while	Wiederholungen mit Zähler, range() und Abbruchbedingung steuern.
6	Listen	Mehrere Werte speichern, per Index lesen, ergänzen und durchlaufen.
7	Zeichenketten	Strings indizieren, zerlegen, normalisieren und Zeichen zählen.
8	Funktionen	Aufgaben in benannte Bausteine zerlegen, Parameter nutzen und Werte zurückgeben.
9	Konzepte verbinden und Eingaben prüfen	Listen, Schleifen, Bedingungen und Funktionen zu robusten Programmen kombinieren.
10	Mini-Projekt	Ein kleines Programm planen, schrittweise bauen, testen und kurz vorstellen.

Tag 1 · Erste Programme: Ausgabe und Syntax

Dein Problemziel

Anweisungen von oben nach unten lesen, Text ausgeben, Kommentare verwenden.

Denkstrategie: Vom Ziel rückwärts denken

Welche Ausgabe soll entstehen? Welche einzelnen Informationen müssen dafür ausgegeben werden?

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
print("Hallo, Welt!")

# Ein Kommentar
print("Python startet!")
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Drei Zeilen

Gib deinen Namen, dein Studienfach und einen freundlichen Gruß in drei getrennten Zeilen aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Drei sichtbare Ausgabezeilen.

Plan / Notizen: _____

LEICHT

2. Ausgabe vorhersagen

Notiere zuerst auf Papier, was drei aufeinanderfolgende print()-Anweisungen ausgeben. Führe den Code danach aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Stimmt deine Vorhersage exakt?

Plan / Notizen: _____

LEICHT

3. Mini-Steckbrief

Erzeuge einen Steckbrief mit Überschrift und vier Angaben. Nutze zunächst nur print().

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Keine Eingabe erforderlich.

Plan / Notizen: _____

Tag 1 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Welche Ausgabe soll entstehen? Welche einzelnen Informationen müssen dafür ausgegeben werden?

Probleme vertiefen

NORMAL

4. Kassenbon

Gib einen kleinen Kassenbon mit drei Artikeln und einer Gesamtsumme aus. Richte ihn lesbar aus.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Die Ausgabe soll wie ein einfacher Beleg wirken.

Plan / Notizen: _____

NORMAL

5. Fehlerdetektiv

Korrigiere fünf vorbereitete Fehler: fehlendes Anführungszeichen, falsche Klammer, falsche Großschreibung, zusätzlicher Buchstabe und fehlende Klammer.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Ändere jeweils nur das Nötigste.

Plan / Notizen: _____

HERAUSFORDERUNG

6. ASCII-Bild

Zeichne mit mehreren print()-Zeilen ein Haus, Tier oder Gesicht aus Tastaturzeichen.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Mindestens 6 Zeilen und ein Kommentar.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 2 · Variablen und Datentypen

Dein Problemziel

Werte speichern, verändern und als int, float, str oder bool einordnen.

Denkstrategie: Werte und Beziehungen erkennen

Welche Größen sind gegeben? Welche Werte werden daraus berechnet? Gib jedem Wert einen verständlichen Namen.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
name = "Mina"
alter = 19
groesse = 1.72
ist_studentin = True

breite = 5
hoehe = 3
flaeche = breite * hoehe
print(flache)
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Vier Typen

Lege je eine Variable für Name, Alter, Körpergröße und Studierendenstatus an. Gib Wert und type(...) aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Erwartet werden str, int, float und bool.

Plan / Notizen: _____

LEICHT

2. Rechteck

Speichere Breite und Höhe. Berechne Fläche und Umfang.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Für 5 und 3: Fläche 15, Umfang 16.

Plan / Notizen: _____

LEICHT

3. Minuten

Wandle 7 Minuten in Sekunden um. Nutze Variablen.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Ergebnis: 420 Sekunden.

Plan / Notizen: _____

Tag 2 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Welche Größen sind gegeben? Welche Werte werden daraus berechnet? Gib jedem Wert einen verständlichen Namen.

Probleme vertiefen

NORMAL

4. Temperatur

Wandle 20 Grad Celsius mit $F = C * 9 / 5 + 32$ in Fahrenheit um.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Ergebnis: 68.0.

Plan / Notizen: _____

NORMAL

5. Satz mit Werten

Baue mit einem f-String einen Satz aus mindestens drei Variablen.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Keine Plus-Zeichen zwischen Text und Zahlen nötig.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Zeit zerlegen

Zerlege 3672 Sekunden mit // und % in Stunden, Minuten und Restsekunden.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: 1 h, 1 min, 12 s.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 3 · Eingabe und Operatoren

Dein Problemziel	input() verwenden, Texte in Zahlen umwandeln und Rechenoperatoren sicher einsetzen.
Denkstrategie: Eingabe - Verarbeitung - Ausgabe	Markiere bei jeder Aufgabe, was eingegeben wird, was damit geschieht und was am Ende sichtbar sein soll.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
name = input("Name: ")
print(f"Hallo, {name}!")

zahl = int(input("Zahl: "))
print(zahl * 2)
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Begrüßung

Frage Name und Wohnort ab und gib eine persönliche Begrüßung aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Eingabe: Ada, Berlin.

Plan / Notizen: _____

LEICHT

2. Zwei Zahlen

Lies zwei ganze Zahlen ein und gib Summe, Differenz und Produkt aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Teste mit 10 und 4.

Plan / Notizen: _____

LEICHT

3. Rechteck interaktiv

Lies Breite und Höhe als float ein und berechne die Fläche.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Teste auch 2.5 und 4.

Plan / Notizen: _____

Tag 3 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Markiere bei jeder Aufgabe, was eingegeben wird, was damit geschieht und was am Ende sichtbar sein soll.

Probleme vertiefen

NORMAL

4. Celsius-Rechner

Lies Celsius ein und gib Fahrenheit aus. Runde zunächst nicht.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: $0\text{ °C} \rightarrow 32\text{ °F}$.

Plan / Notizen: _____

NORMAL

5. Sekunden-Rechner

Lies eine Sekundenzahl ein und zerlege sie wie an Tag 2.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: $3672 \rightarrow 1:01:12$.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Mini-Rechner

Lies zwei Zahlen ein und zeige +, -, *, / sowie Rest %. Weise bei der Division im Ausgabetext auf den Divisor hin.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Teste 10 und 4; Division 2.5.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 4 · Bedingungen: if, elif, else

Dein Problemziel

Vergleiche formulieren und genau einen passenden Programmzweig ausführen.

Denkstrategie: Fälle vollständig sammeln

Notiere alle möglichen Fälle, bevor du entscheidest. Prüfe besonders die Grenzen zwischen zwei Fällen.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
zahl = int(input("Zahl: "))
if zahl > 0:
    print("positiv")
elif zahl < 0:
    print("negativ")
else:
    print("null")

if alter >= 18 and hat_ticket:
    print("Einlass")
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Positiv, negativ oder null

Ordne eine eingegebene Zahl einer der drei Kategorien zu.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Teste -2, 0 und 8.

Plan / Notizen: _____

LEICHT

2. Gerade oder ungerade

Nutze den Restoperator, um eine ganze Zahl zu prüfen.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: 8 gerade; 7 ungerade.

Plan / Notizen: _____

LEICHT

3. Größere Zahl

Lies zwei Zahlen ein und gib die größere aus. Behandle Gleichheit.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: 5 und 5 → gleich groß.

Plan / Notizen: _____

Tag 4 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Notiere alle möglichen Fälle, bevor du entscheidest. Prüfe besonders die Grenzen zwischen zwei Fällen.

Probleme vertiefen

NORMAL

4. Notenstufen

Ordne Punkte 90–100, 80–89, 70–79, 60–69 und darunter Kategorien zu. Prüfe zuerst ungültige Werte.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: 101 und -1 sind ungültig.

Plan / Notizen: _____

NORMAL

5. Ticketpreis

Bestimme anhand des Alters: unter 6 kostenlos, bis 17 ermäßigt, bis 64 regulär, sonst Seniorentarif.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Prüfe Grenzwerte 5/6/17/18/64/65.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Schaltjahr

Ein Jahr ist Schaltjahr, wenn es durch 4 teilbar ist, außer durch 100 - es sei denn, auch durch 400.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: 1900 nein, 2000 ja, 2024 ja.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 5 · Schleifen: for und while

Dein Problemziel

Wiederholungen mit Zähler, range() und Abbruchbedingung steuern.

Denkstrategie: Wiederholungen beschreiben

Vervollständige den Satz: „Wiederhole _____, solange / für jedes _____.“
Bestimme, was sich pro Durchlauf ändert.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
for zahl in range(1, 6):
    print(zahl)

rest = 3
while rest > 0:
    print(rest)
    rest -= 1
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Zählen

Gib mit for die Zahlen 1 bis 10 aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Achte auf die exklusive Obergrenze von range().

Plan / Notizen: _____

LEICHT

2. Gerade Zahlen

Gib alle geraden Zahlen von 2 bis 20 aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Nutze range() mit Schrittweite.

Plan / Notizen: _____

LEICHT

3. Countdown

Zähle mit while von 10 bis 1 und gib danach Los! aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Die Schleife muss enden.

Plan / Notizen: _____

Tag 5 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Vervollständige den Satz: „Wiederhole _____, solange / für jedes _____.“
Bestimme, was sich pro Durchlauf ändert.

Probleme vertiefen

NORMAL

4. Einmaleins

Lies n ein und gib $n * 1$ bis $n * 10$ aus.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Für $n=3$ endet die Tabelle bei 30.

Plan / Notizen: _____

NORMAL

5. Summe 1 bis n

Berechne die Summe mit Schleife, nicht mit einer Formel.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: $n=5 \rightarrow 15$.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Passwortversuche

Erlaube höchstens drei Eingaben. Bei richtigem Passwort endet die Schleife sofort, sonst folgt eine Sperrmeldung.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Teste Erfolg beim 1. und 3. Versuch sowie dreimal falsch.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 6 · Listen

Dein Problemziel

Mehrere Werte speichern, per Index lesen, ergänzen und durchlaufen.

Denkstrategie: Viele Werte systematisch untersuchen

Kläre, was für jeden Wert gleich abläuft. Starte bei Minimum oder Maximum nicht automatisch mit 0.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
zahlen = [4, 8, 2, 9]
print(zahlen[0])
zahlen.append(5)

for zahl in zahlen:
    print(zahl)
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Elemente ausgeben

Lege eine Liste mit fünf Lieblingssessen an und gib jedes Element einzeln aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Nutze eine Schleife.

Plan / Notizen: _____

LEICHT

2. Liste verändern

Füge ein Element mit `append()` hinzu und entferne eines mit `remove()`.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Gib vorher und nachher aus.

Plan / Notizen: _____

LEICHT

3. Summe bilden

Addiere alle Zahlen einer Liste mit eigener Schleife.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Vergleiche erst danach mit `sum()`.

Plan / Notizen: _____

Tag 6 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Kläre, was für jeden Wert gleich abläuft. Starte bei Minimum oder Maximum nicht automatisch mit 0.

Probleme vertiefen

NORMAL

4. Durchschnitt

Berechne Summe / len(liste). Behandle eine leere Liste mit einer Bedingung.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: [2,4,6] → 4.0.

Plan / Notizen: _____

NORMAL

5. Maximum selbst finden

Starte mit dem ersten Listenelement und vergleiche alle weiteren. max() ist hier tabu.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: [-5,-2,-9] → -2.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Notenstatistik

Ermittle aus fünf Punktzahlen Durchschnitt, Maximum, Minimum und Anzahl bestandener Werte ab 50.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Nutze mindestens eine Schleife.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 7 · Zeichenketten

Dein Problemziel

Strings indizieren, zerlegen, normalisieren und Zeichen zählen.

Denkstrategie: Eingaben normalisieren

Entscheide, ob Großschreibung oder Leerzeichen die Bedeutung verändern sollen. Bereite den Text vor dem Vergleich auf.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
text = "Python"
print(text[0])
print(text[-1])
print(text[1:4])

wort = input("Wort: ").strip().lower()
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Zeichen einzeln

Gib jedes Zeichen eines eingegebenen Wortes in einer eigenen Zeile aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Python ergibt 6 Zeilen.

Plan / Notizen: _____

LEICHT

2. Länge und Rand

Gib Länge, erstes und letztes Zeichen aus. Prüfe vorher, ob der Text leer ist.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Leere Eingabe darf nicht abstürzen.

Plan / Notizen: _____

LEICHT

3. Groß und klein

Zeige denselben Text mit upper(), lower() und title().

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Beobachte Leerzeichen.

Plan / Notizen: _____

Tag 7 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Entscheide, ob Großschreibung oder Leerzeichen die Bedeutung verändern sollen. Bereite den Text vor dem Vergleich auf.

Probleme vertiefen

NORMAL

4. Vokale zählen

Zähle a, e, i, o, u unabhängig von Großschreibung.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Informatik → 4.

Plan / Notizen: _____

NORMAL

5. Text umdrehen

Gib einen Text rückwärts aus - zuerst mit Slicing, freiwillig zusätzlich mit Schleife.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Python → nohtyP.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Palindrom

Ignoriere Groß-/Kleinschreibung und äußere Leerzeichen. Vergleiche normalisierten Text mit seiner Umkehrung.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Anna → Palindrom; Python → keines.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 8 · Funktionen

Dein Problemziel

Aufgaben in benannte Bausteine zerlegen, Parameter nutzen und Werte zurückgeben.

Denkstrategie: Teilprobleme benennen

Wenn du einen Schritt in einem kurzen Satz benennen kannst, kann daraus eine Funktion werden.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
def begruesse(name):
    print(f"Hallo, {name}!")

def addiere(a, b):
    return a + b
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Quadrat

Schreibe `quadrat(zahl)`, das das Quadrat zurückgibt.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: `quadrat(4)` → 16.

Plan / Notizen: _____

LEICHT

2. Begrüßung

Schreibe `begruesse(name)`, das einen Begrüßungstext zurückgibt.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Rückgabe statt Eingabe in der Funktion.

Plan / Notizen: _____

LEICHT

3. Gerade prüfen

Schreibe `ist_gerade(zahl)` mit bool-Rückgabe.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: `ist_gerade(7)` → False.

Plan / Notizen: _____

Tag 8 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Wenn du einen Schritt in einem kurzen Satz benennen kannst, kann daraus eine Funktion werden.

Probleme vertiefen

NORMAL

4. Maximum

Schreibe `maximum(a, b)`, ohne `max()` zu verwenden. Gleichheit soll erlaubt sein.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: `maximum(-2,-5) → -2`.

Plan / Notizen: _____

NORMAL

5. Temperaturfunktion

Schreibe `celsius_zu_fahrenheit(c)`. Teste mindestens drei Werte.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: `0 → 32`; `100 → 212`.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Rechner zerlegen

Erstelle `addieren`, `subtrahieren`, `multiplizieren` und `dividieren`. Bei Division durch 0 soll eine verständliche Rückgabe entstehen.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Jede Funktion separat testen.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 9 · Konzepte verbinden und Eingaben prüfen

Dein Problemziel

Listen, Schleifen, Bedingungen und Funktionen zu robusten Programmen kombinieren.

Denkstrategie: Fehler als Testfälle nutzen

Plane ungültige Eingaben bewusst ein. Ein robustes Programm erklärt den Fehler und bietet einen nächsten Schritt an.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
try:
    alter = int(input("Alter: "))
except ValueError:
    print("Bitte eine ganze Zahl eingeben.")

while True:
    befehl = input("> ").strip().lower()
    if befehl == "ende":
        break
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Sichere Ganzzahl

Wiederhole die Eingabe, bis `int(...)` gelingt. Gib danach die Zahl aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Teste Text, Dezimalzahl und ganze Zahl.

Plan / Notizen: _____

LEICHT

2. Wortliste

Lies Wörter ein, bis `ende` eingegeben wird. Gib danach die Liste aus.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: `ende` gehört nicht zur Liste.

Plan / Notizen: _____

LEICHT

3. Menü

Zeige ein Menü wiederholt und beende nur bei Auswahl 0. Unbekannte Auswahl erhält eine Meldung.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Teste 1, 9 und 0.

Plan / Notizen: _____

Tag 9 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Plane ungültige Eingaben bewusst ein. Ein robustes Programm erklärt den Fehler und bietet einen nächsten Schritt an.

Probleme vertiefen

NORMAL

4. Einkaufsliste

Menü: hinzufügen, anzeigen, entfernen, beenden. Verhindere Fehler beim Entfernen unbekannter Einträge.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Liste bleibt während der Schleife erhalten.

Plan / Notizen: _____

NORMAL

5. Zahlen raten

Nutze zunächst eine feste Geheimzahl. Gib zu klein, zu groß oder richtig aus und zähle Versuche.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Geheimzahl 7; mehrere Testwege.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Quiz-Runde

Speichere Fragen und Antworten in parallelen Listen oder als Liste von Paaren. Zähle Punkte und gib am Ende eine Quote aus.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Mindestens 5 Fragen; Großschreibung ignorieren.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Tag 10 · Mini-Projekt

Dein Problemziel	Ein kleines Programm planen, schrittweise bauen, testen und kurz vorstellen.
Denkstrategie: Klein anfangen, dann erweitern	Baue zuerst die kleinste lauffähige Version. Teste sie. Ergänze danach immer nur eine Erweiterung.

Kleine Werkzeugbox

Schau hier nur nach, wenn du für deinen geplanten Schritt Python-Schreibweise brauchst.

```
# 1. Ziel in einem Satz
# 2. Eingaben und Ausgaben
# 3. Teilaufgaben als Funktionen
# 4. Testfälle

def main():
    ...

main()
```

Aufgabenwerkstatt · Einstieg

LEICHT

1. Projekt wählen

Wähle Quiz, To-do-Liste, Rechner, Zahlenraten oder Schere-Stein-Papier. Formuliere Ziel und Mindestfunktionen.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Das Projekt ist in 60–90 Minuten machbar.

Plan / Notizen: _____

LEICHT

2. Ablauf skizzieren

Schreibe Pseudocode: Start, Eingaben, Verarbeitung, Ausgabe, Wiederholung, Ende.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Noch kein Python-Code nötig.

Plan / Notizen: _____

LEICHT

3. Testplan

Notiere mindestens fünf Testfälle, darunter Grenz- oder Fehleingaben.

Denk zuerst: Schreibe zuerst ein konkretes Beispiel mit Eingabe und gewünschter Ausgabe auf.

Prüfe deine Idee: Eingabe und erwartete Ausgabe festhalten.

Plan / Notizen: _____

Tag 10 · Aufgabenwerkstatt (Fortsetzung)

Erinnere dich

Baue zuerst die kleinste lauffähige Version. Teste sie. Ergänze danach immer nur eine Erweiterung.

Probleme vertiefen

NORMAL

4. Mindestversion bauen

Implementiere eine lauffähige Version mit mindestens einer Schleife, einer Bedingung, einer Liste und zwei eigenen Funktionen.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Kleine funktionierende Version zuerst.

Plan / Notizen: _____

NORMAL

5. Benutzbarkeit verbessern

Ergänze klare Prompts, verständliche Fehlermeldungen und eine saubere Abschlussausgabe.

Denk zuerst: Notiere zuerst: Eingabe → Verarbeitung → Ausgabe. Zerlege die Verarbeitung in kleine Schritte.

Prüfe deine Idee: Eine andere Person kann es ohne Erklärung bedienen.

Plan / Notizen: _____

HERAUSFORDERUNG

6. Erweiterung

Ergänze genau eine sinnvolle Funktion: Zufall, Highscore, mehrere Runden, Statistik oder Schwierigkeitsgrad.

Denk zuerst: Lege zuerst eine Mindestversion fest. Notiere danach einen Normalfall und zwei Grenz- oder Fehlerfälle.

Prüfe deine Idee: Die Mindestversion muss weiterhin funktionieren.

Plan / Notizen: _____

Dein Exit-Ticket

Welche Denkstrategie hat dir heute geholfen? _____

Welchen Testfall würdest du einer anderen Person empfehlen? _____

Abschlussprojekt · Checkliste

Nutze diese Checkliste am Tag 10. Ein kleines vollständiges Projekt ist besser als ein großes unfertiges.

Mindestumfang

Eine wiederholbare Hauptschleife · mindestens eine if/elif/else-Entscheidung ·
mindestens eine Liste · mindestens zwei eigene Funktionen · verständliche
Ein- und Ausgaben.

Vor der Abgabe

- ☐ Programm startet ohne Fehler.
- ☐ Alle Menüwege wurden mindestens einmal getestet.
- ☐ Ungültige Eingaben führen nicht sofort zum Absturz.
- ☐ Variablen und Funktionen haben verständliche Namen.
- ☐ Doppelte Logik wurde möglichst in Funktionen ausgelagert.
- ☐ Die Ausgabe erklärt, was als Nächstes möglich ist.
- ☐ Eine andere Person hat das Programm ausprobiert.

Kurzvorstellung (2 Minuten)

1. Was macht das Programm? 2. Welche Eingabe wird erwartet? 3. Zeige einen normalen Ablauf. 4. Nenne eine Schwierigkeit und deine Lösung.

Python-Cheatsheet

Die wichtigsten Bausteine dieses Vorkurses auf wenigen Seiten.

Ausgabe, Kommentare und Variablen

```
print("Hallo")
# Kommentar
name = "Mira"
alter = 20
print(f"{name} ist {alter} Jahre alt.")
```

Datentypen und Umwandlung

```
ganze_zahl = 12 # int
kommazahl = 3.5 # float
text = "Python" # str
wahrheitswert = True # bool

int("12") float("3.5") str(12) type(text)
```

Eingabe

```
name = input("Name: ") # liefert str
alter = int(input("Alter: ")) # Umwandlung zu int
```

Rechnen und Vergleichen

Operator	Bedeutung	Beispiel
+ - * /	Grundrechenarten	$7 / 2 \rightarrow 3.5$
//	Ganzzahldivision	$7 // 2 \rightarrow 3$
%	Rest	$7 \% 2 \rightarrow 1$
**	Potenz	$2 ** 3 \rightarrow 8$
== !=	gleich / ungleich	$x == 5$
< <= > >=	Vergleiche	$alter \geq 18$
and or not	Logik	$x > 0 \text{ and } x < 10$

Python-Cheatsheet · Steuerung

Bedingungen

```
if punkte >= 90:
    print("sehr gut")
elif punkte >= 50:
    print("bestanden")
else:
    print("noch nicht bestanden")
```

for-Schleife und range()

```
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 11, 2): # 2, 4, 6, 8, 10
    print(i)
```

while-Schleife

```
versuche = 3
while versuche > 0:
    print(versuche)
    versuche -= 1
```

Einrückung

Nach if, elif, else, for, while und def steht ein Doppelpunkt. Der zugehörige Block wird gleichmäßig eingerückt - üblich sind vier Leerzeichen.

Listen

```
namen = ["Ada", "Linus"]
namen.append("Grace")
print(namen[0])
print(len(namen))

for name in namen:
    print(name)
```

Nützliche Listenoperationen

```
liste.append(x) # hinzufügen
liste.remove(x) # ersten passenden Wert entfernen
x in liste # Mitgliedschaft prüfen
len(liste) # Anzahl
sum(zahlen) # Summe
min(zahlen), max(zahlen)
```

Python-Cheatsheet · Strings und Funktionen

Strings

```
text = input("Text: ").strip()
print(text.lower())
print(text.upper())
print(text[0]) # erstes Zeichen
print(text[-1]) # letztes Zeichen
print(text[::-1]) # rückwärts
print(len(text))
```

Funktionen

```
def flaeche(breite, hoehe):
    ergebnis = breite * hoehe
    return ergebnis

print(flaeche(5, 3))
```

Fehler bei Eingaben abfangen

```
try:
    zahl = int(input("Ganze Zahl: "))
except ValueError:
    print("Das war keine ganze Zahl.")
```

Häufige Fehlermeldungen

Meldung	Oft bedeutet das
SyntaxError	Schreibfehler, fehlender Doppelpunkt, Klammer oder Anführungszeichen
IndentationError	Einrückung fehlt oder ist uneinheitlich
NameError	Variablen- oder Funktionsname ist unbekannt / anders geschrieben
TypeError	Operation passt nicht zu den Datentypen
ValueError	Umwandlung ist grundsätzlich möglich, aber dieser Wert passt nicht
IndexError	Listen- oder String-Index liegt außerhalb des gültigen Bereichs

Deutsch im Programmierunterricht

Du musst englische Codewörter nicht übersetzen: Gesprochen wird deutsch, Python bleibt Python. Hilfreich sind feste Formulierungen, die du wiederholt verwendest.

Begriffe

Englisch / Code	Gute deutsche Formulierung
variable	Variable; speichert einen Wert
assignment	Zuweisung
statement	Anweisung
expression	Ausdruck
condition	Bedingung
branch	Programmzweig / Fall
loop / iteration	Schleife / Schleifendurchlauf
list / index	Liste / Index
string	Zeichenkette; im Gespräch oft auch String
function / parameter	Funktion / Parameter
argument	Argument bzw. eingesetzter Wert
return value	Rückgabewert
bug / debugging	Fehler / Fehlersuche bzw. Debugging
exception	Ausnahme / Laufzeitfehler
indentation	Einrückung

Nützliche Tutor-Sätze

- Was soll das Programm als Eingabe bekommen?
- Was soll am Ende ausgegeben werden?
- Welche Teilaufgaben erkennst du?
- Sage zuerst in eigenen Worten, was die Schleife wiederholt.
- Welche Werte sollten wir zum Testen wählen?
- Lies die letzte Zeile der Fehlermeldung laut vor.
- Was hast du erwartet - und was ist tatsächlich passiert?
- Zeige mir die kleinste Stelle, an der das Ergebnis falsch wird.

Weiterlernen: eine sinnvolle Auswahl

CS50P noch einmal komplett ansehen? Wahrscheinlich nicht. Für die Vorbereitung dieses zweiwöchigen Vorkurses reicht ein gezielter Auffrischungsdurchgang: Functions & Variables, Conditionals, Loops und Exceptions. Sieh die Vorlesungen abschnittsweise und löse jeweils ein bis zwei kleine Aufgaben selbst. Libraries, Unit Tests, File I/O, Regular Expressions und OOP gehen deutlich über den hier geplanten Kern hinaus.

Empfehlung

Plane 4 kurze Wiederholungsblöcke à 45–75 Minuten. Schreibe bei jedem Block ein Mini-Programm ohne Vorlage. Für Tutorinnen und Tutoren ist aktives Erklären wichtiger als passives Wiederansehen.

Deutschsprachige Ressourcen

openHPI: „Python - schnell und intensiv Programmieren lernen“

Deutschsprachiger Selbstlernkurs mit Übungen und automatischem Feedback. Nutze besonders Abschnitte zu Datentypen, Funktionen, Bedingungen, Schleifen und Fehlerbehandlung.

<https://open.hpi.de/learn/python2024?locale=de>

python-kurs.eu

Frei zugängliches deutsches Nachschlagewerk mit vielen Beispielen und Übungen. Gut, um Formulierungen und Fachwortschatz für einzelne Unterrichtsthemen vorzubereiten.

<https://www.python-kurs.eu/>

CS50P

Didaktisch stark und sehr übungsorientiert, aber englisch. Nutze es gezielt für deine fachliche Auffrischung; das deutsche Vokabular trainierst du parallel mit den beiden Quellen oben.

<https://cs50.harvard.edu/python/>

So trainierst du die deutsche Erklärung

Wähle pro Kurstag fünf Begriffe aus der Begriffstabelle. Erkläre den Tagescode zwei Minuten laut auf Deutsch, ohne ihn nur vorzulesen. Nimm dich einmal auf und prüfe: Sind Eingabe, Verarbeitung und Ausgabe klar? Verwendest du dieselben Begriffe konsistent?

Stand der Links: August 2026.